# Paiflow Developer Guide

Updated Oct 9, 2026

Build your first payment integration: a customer's XLM payment automatically splits 90% to a shop and 10% to a community fund. Start with a working flow, prove its payout, then connect your own app.

## 01 Before you begin

This walkthrough is event-neutral. Follow your participant brief and organisers' instructions for when preparation, deployment and app integration are allowed.

Paiflow is the money logic behind your app. Your app handles who pays and why; a deployed flow handles when money moves and where it goes. One deployment serves one business or group and can receive payments from many customers.

| You need | Purpose |
| --- | --- |
| Paiflow | Build, deploy and inspect your payment flow |
| A full/team Paiflow account | Create your own deployment API token |
| Freighter | Connect wallets and approve testnet transactions |
| Node.js 22, pnpm 10 and git | Run the supplied starter app |
| Dedicated testnet wallets | Deploy, pay and receive the split |

> **Sandbox or team account?** The sandbox supports practice using its available features, including flows and test payments. Sandbox accounts cannot create deployment API tokens or connect their deployments to your app through the partner API. Use a full/team account for the integration steps.

## 02 Prepare your testnet wallets

1. Install Freighter in the browser you will use for Paiflow and your local app.
2. Create dedicated testnet accounts for the deployer/customer, shop and community fund. The deployer may also act as the customer for this first walkthrough; keep recipient accounts distinct.
3. Switch Freighter to **Stellar Testnet**. Fund each account with testnet XLM using Freighter's Friendbot prompt or Stellar Lab.
4. Copy each account's public address, starting with `G`. Check that all accounts exist and the paying wallet has enough XLM for the payment and transaction fees.

XLM needs no trustline. Friendbot supplies testnet XLM, not USDC. See Stellar's Freighter testnet setup guide for wallet setup.

> **Keep wallet secrets in your wallet.** Never paste a secret key or recovery phrase into Paiflow, your app, an AI prompt or a repository. Testnet funds have no real-money value.

# 03 Build a Receive → Split flow

1. Open Paiflow and sign in with your full/team account when integration is permitted.
2. Create a flow, using **On Receive** as its trigger and **Split** as its action. Connect On Receive to Split. Keep Dev mode off for this example.
3. Select native **XLM** as the flow's asset. Remove unrelated blocks or default recipients from the example graph.
4. Configure two percentage recipients using your actual funded public addresses:

| Recipient | Percentage | Expected share of 1 XLM |
|---|---|---|
| **Shop** | 90% | 0.9 XLM |
| **Community fund** | 10% | 0.1 XLM |

The shares must total 100%. The UI uses percentages; in the graph/API reference, 90% is `9000` basis points and 10% is `1000`. Replace every placeholder address before deploying.

> **Checkpoint:** your graph has one trigger, a connected Split action, XLM throughout and two real recipients. You are modelling one shop's payment rules, not a separate contract for each customer.

# 04 Review and deploy

1. Open the deployment review and read the **English preview**. Confirm that it describes receiving XLM and paying your two recipients at the intended percentages.
2. Connect the funded deployer wallet. Confirm Freighter is on Testnet.
3. Start deployment. In Freighter, review the network, source account, requested contract operations and displayed fee before confirming.
4. Wait for the deployment to report **CONFIRMED**, then open its deployment page.

Copy the deployment's UUID. It is different from the flow ID and the Stellar contract address beginning with `C`. API calls use the deployment UUID.

> **Checkpoint:** you have a confirmed deployment with the correct asset and recipients. Deployment alone does not prove a payment succeeded; make a test payment next. If confirmation is uncertain, check the existing deployment/transaction before starting another deployment.

# 05 Make a payment through the hosted page

1. Open the deployment's hosted payment page or QR link. For this Receive flow, its path is `/trigger/YOUR_DEPLOYMENT_UUID` on the Paiflow site.
2. Connect your funded customer wallet. If you changed accounts in Freighter, check that the app shows the intended customer's public address.
3. Enter **1 XLM**, review the payment and approve the Testnet transaction in Freighter.
4. Wait for confirmation. Inspect the deployment's live activity and transaction evidence for the split payout.

For this example, the payment should route **0.9 XLM to the shop and 0.1 XLM to the community fund**. Fees are additional to the customer's 1 XLM payment. If a recipient is also the paying wallet, its net balance change will include both the payment and fees; use the transaction's payout evidence to check the split.

**Checkpoint:** save the payment transaction hash and verify its expected payouts. A slow live feed is not proof of failure; check the existing transaction before paying again. You can look up the hash on Stellar Expert Testnet.

# 06 Create your deployment API token

1. Return to your deployment page while signed in as its owner.
2. Open **API access**, create a token and copy it immediately. The full token is shown once.
3. Keep the token with your confirmed deployment UUID for the starter's server configuration.

| Value | How to recognise it |
|---|---|
| **Deployment UUID** | UUID for the confirmed deployment; not the flow ID or contract address |
| **API token** | Begins with `pfk_`; belongs to this deployment |
| **Customer address** | Public `G…` address; comes from the customer's connected wallet |

**Keep the API token server-side.** Do not put it in browser code, `NEXT_PUBLIC_` variables, screenshots or public repositories. If exposed, revoke it and issue a replacement. Customers approve their own payments in their wallets; your server never needs their secret keys.

# 07 Run the starter in preparation mode

Use the Paiflow hackathon starter. Run these commands in a terminal:

```
git clone https://github.com/artisam-paiflow/paiflow-hackathon-starter.git
cd paiflow-hackathon-starter
corepack enable
pnpm install
cp .env.example .env.local
pnpm dev
```

Open `http://localhost:3000`. These copy commands work in bash; on Windows you can copy `.env.example` to `.env.local` in your editor instead. Keep `PAIFLOW_MODE=prepare` while preparing non-payment screens or while event rules require it.

Preparation mode disables wallet actions, payments and event polling. It makes no upstream Paiflow requests. Payment, payout and event routes return `403 PREPARATION_MODE`; that is expected in this mode.

> **Checkpoint:** the app opens locally and you can work on its non-payment screens. Keep the starter's mode guards when you customise it.

# 08 Configure your own deployment

When integration is permitted, stop the dev server. Edit `.env.local` using your deployment and token:

```
PAIFLOW_MODE=team
PAIFLOW_BASE_URL=https://beta.paiflow.xyz
PAIFLOW_DEPLOYMENT_ID=YOUR_CONFIRMED_DEPLOYMENT_UUID
PAIFLOW_API_TOKEN=YOUR_DEPLOYMENT_API_TOKEN
```

Replace both placeholders with the real values. If your organiser supplies a different API origin, use that origin for `PAIFLOW_BASE_URL`. The starter's copied example may have an older default; set it explicitly. Restart with `pnpm dev` after changing configuration.

| Mode | Meaning |
|---|---|
| `prepare` | Non-payment screens only; default when mode is unset |
| `team` | Your deployment and server-side token |
| `demo` | Optional shared XLM demo, where enabled and permitted |

> Use **team** mode for this walkthrough. An empty token does not automatically select demo mode. Shared-demo transactions belong to a different deployment and do not demonstrate your own integration.

Ensure `.env.local` is ignored by git. Do not commit it. Missing or invalid team configuration disables integration with an error; fix the UUID, token or origin rather than bypassing the guards.

# 09 Make a payment from your app

1. Open the local starter in the browser with Freighter installed. Confirm team mode is configured and the deployment uses XLM.
2. Select **Connect Freighter** and approve access to the local site. Connecting to Paiflow does not automatically grant access to your local app.
3. Confirm the customer address and Testnet network. Enter **1** in the ordinary **Pay** amount field and select **Pay**. Do not use the per-payment recipient form for this fixed Split example.
4. Review the Testnet signing request in Freighter and confirm. The starter prepares through its server, Freighter signs, and the server submits.
5. Check for **SUCCESS**, then verify the expected 0.9/0.1 XLM payout. Save the transaction hash.

The amount field uses decimal units of your flow's input asset. Internally, 1 XLM is the integer string `"10000000"`. Let `lib/amount.ts` handle conversion with bigint; avoid floating-point money arithmetic.

> **If pending:** use **Check again** with the same signed transaction. Keep the tab open. A timeout or submission `502` may mean it was already broadcast; do not prepare a duplicate payment because the result is uncertain. If you switch customer accounts, select **Refresh wallet** before paying again.

# 10 Replace the screens with your idea

Keep the working payment path while changing the customer experience:

| File | Role |
|---|---|
| `components/starter-app.tsx` | Customer UI; replace with your checkout, dues or donation experience |
| `app/page.tsx` | Server page wrapper and configuration readiness |
| `lib/paiflow.ts` | Server-only Paiflow client and mode guards |
| `lib/amount.ts` | Exact decimal-to-stroops conversion |

> The request path is **browser → your server → Paiflow → customer wallet signature → your server → Paiflow submission**. Return unsigned preparation data to the browser for wallet signing; keep the bearer token on the server. Treat HTTP 200 as a transport result, not payment confirmation: inspect transaction status and the expected payout.

Keep the deployment UUID and transaction hash with your app's payment record. Associate feed activity by transaction hash and deduplicate by `eventId`. Before exposing privileged payout or early-release routes, add your app's own authorisation.

For subscriptions, schedules, escrow, webhooks or changing recipients per payment, use the developer context guide. Those features have different funding, consent and execution requirements; they are not interchangeable with this simple Receive flow.

# 11 Troubleshoot your first integration

| Symptom | What to check |
|---|---|
| **Freighter missing or not connecting** | Install/unlock it in this browser; approve this site's access; check the active account and Testnet network |
| **No payment controls or `PREPARATION_MODE`** | Set server-only `PAIFLOW_MODE=team` when permitted; restart the app |
| **Configuration error, `401` or `404`** | Check the origin, confirmed deployment UUID and active token belonging to that deployment |
| **Insufficient funds or missing account** | Friendbot-fund the payer and recipients; leave XLM for fees and reserves |
| **Missing trustline after switching to USDC** | Every receiving wallet needs the exact USDC asset trustline; the payer also needs USDC funds |
| **`422` validation error** | Read the message; check asset, amount, recipients and trigger type |
| **`429` rate limit** | Wait for `Retry-After`; avoid repeated clicks and excessive polling |
| **`PENDING`, timeout or submission `502`** | Preserve and resubmit the same signed envelope; check finality before creating a new payment |
| **`FAILED`** | Read the confirmed failure and fix its cause before preparing another payment |
| **Payment confirmed, feed not updated** | Verify by transaction hash; allow the feed to catch up instead of paying again |

A confirmed deposit can leave funds held in other flow types. For this Split example, verify the two expected payouts. When asking for help, include the deployment UUID, transaction hash, error code/message and `x-request-id` if available. Omit secrets and signed envelopes.

# 12 Check your result and go further

Your first integration is ready when you have:

- A confirmed XLM Receive → Split deployment using your real recipient addresses.
- A hosted-page payment and a starter-app payment with verifiable payouts.
- The deployment UUID and payment transaction hashes saved.
- Server-only API credentials, with no wallet secrets in your app.
- A clear way to display success, pending and failure to the customer.

Use these references when extending it:

- Starter repository — setup and working payment plumbing.
- Paiflow developer context guide — blocks, API shapes, constraints and worked examples.
- Freighter Testnet setup — wallet creation, Testnet selection and funding.
- Stellar Expert Testnet — inspect your transaction evidence.

For API details, see the repository's `docs/hackathon/developer-api-quickstart.md` and `docs/api/README.md`, or the target instance's `/api/v1/openapi.json`. Keep event rules, deadlines and submission requirements in your participant brief.